The League — how our bot trains itself

Patient Wolves, Battlecode 2026a, branch `league`

October 2026

Quick start

0. Set up a PC once (a teammate's desktop, your laptop: any Windows PC). The person needs push access to the GitHub repo, because results are shared by pushing to `league`:

```
git clone -b league https://github.com/Ahmed-Abdul-Rahim/battlecode-2026a.git
cd battlecode-2026a
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1
```

Then open PowerShell in the repo folder and activate the environment once per window:

```
.\.venv\Scripts\Activate.ps1
```

1. Start the work (each runs in the background until you stop it):

```
scripts\train.ps1 start                          # any PC: tune the numbers per map
scripts\train.ps1 experiments                    # any PC: test the queued ideas
scripts\train.ps1 ml                             # optional: data games + network
    training
scripts\train.ps1 start --lead --auto-submit     # the ONE lead PC only: ladder + uploads
```

Which to run: on a teammate's PC or a laptop, `start` and `experiments`. Two processes per PC is enough: each already uses every core but two, so a third only shares the same CPU. Only one PC in the team may use `--lead`; `--auto-submit` also needs `cambc login`. The upload rules are in §5.

Stop before you shut down or carry a laptop away (unfinished work returns to the shared queue):

```
scripts\train.ps1 stop
scripts\train.ps1 experiments stop
scripts\train.ps1 ml stop
```

Laptops: keep them plugged in and set "when I close the lid" to *Do nothing* (Control Panel, Power Options), or a sleeping laptop holds its claimed experiment for up to 6 hours. Each PC is known by its computer name; if two share one, set `$env:LEAGUE_HOST = "laptop"` before starting. **After a change to `harness/`** (the Python), restart: `stop`, then `start` and `experiments` again. Bot code and new queue items reach every PC by themselves.

More PCs = faster, not linearly: an 8-core laptop adds about half of what a 16-core desktop does, and each experiment still runs whole on one PC; the queue just drains faster.

2. See how the latest model is doing:

```
scripts\train.ps1 status        # version, win rate per top team, losses
scripts\train.ps1 logs          # live log (Ctrl+C to leave)
scripts\train.ps1 bench         # new test vs every mimic, every map
scripts\train.ps1 review        # full report: league\review\<date>.md
```

3. Watch it play (opens the official viewer in your browser):

```
scripts\train.ps1 replay                    # our worst team + map
scripts\train.ps1 replay nyx lanes          # vs Nyx's mimic on lanes
scripts\train.ps1 replay nyx lanes --evolved   # vs the harder Nyx
```

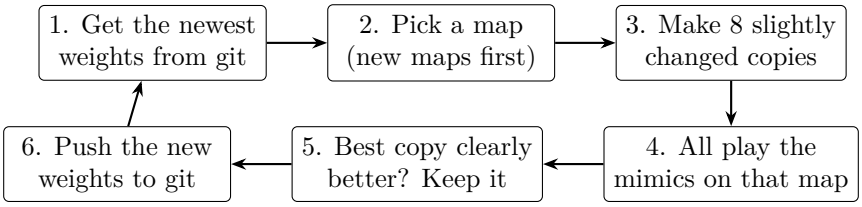
It plays two games (one from each side) and prints win/loss and gems. The viewer opens the first game, and the command for the second is printed. Stop the viewer with Ctrl+C. Real ladder games: `cambc matches download <matchId> -o x.repl26a`, then `cambc view x.repl26a`.

1 The idea in five sentences

1. We copy the 10 best teams on the ladder into practice bots, called **mimics**.
2. Our bot plays thousands of games against these mimics on every map.
3. After each batch of games we try small changes to our bot's numbers, and keep the changes that win more.
4. When we lose on the real ladder, the bot trains on that map first, and a better version can be uploaded.
5. Anything the training cannot fix is written in a report for a human to look at.

Every teammate's PC runs the same program (`harness/league.py`). They share the results through git.

2 One training round



A round takes about 4 minutes. Every third round, we train one of the *mimics* instead of our bot (see §4).

Which map gets trained. In this order:

1. **Maps that were never trained** (new maps, or old ones still on default numbers). Each gets 4 rounds first, and maps we lost on the ladder go first.
2. **The oldest ladder loss of the version that is live now**, unless its map already had 6 rounds since that loss. After that it goes to the human review.
3. **Where we keep losing:** a weighted pick by "map pressure", meaning recent ladder losses on that map (a loss counts less every day, and double if the live version lost it), times how badly we do on that map locally. Every map keeps a small share, so none is forgotten.

When a new version is uploaded, the open losses are marked *superseded*: they leave the queue, because they were losses of the old version, but they stay in the review and still count in map pressure. So training never runs out of work and never replays an endless queue. Every map in the platform's pool always has an entry: a new map starts with the default numbers until training improves them. `train status` checks that the counts agree (*fingerprint 21/21, geometry 21/21, in pack 21/21, trained N/21*). It also lists each untrained map with its rounds so far, e.g. *hooks 2/4*. A map without a fingerprint is never trained, because the bot couldn't load its numbers, and `status` reports it as OUT OF SYNC.

3 What the bot learns

Code vs numbers

Our bot is hand-written Rust code. The code decides *how* to do things: find a path, attack, buy an item, deliver gems. Many of its decisions compare options with a formula, and those formulas contain numbers. Training never touches the code. It only changes these **169 numbers** (67 of them belong to features that are still switched off, so training leaves them alone).

Each map has its own set of 169 numbers. On a small map with camps close together, walking distance matters less than on a big map. On a map where fights happen early, retreating sooner may be better. A map we never trained on uses the default numbers in `bots/main/src/learned.rs`.

The numbers come in five groups.

1. W — 18 dials on our own settings

While writing the bot we picked many settings by hand, such as "retreat at 35% HP". Each W number *multiplies* one such setting. 1.0 means "as we wrote it", 1.2 means 20% more, and 0.8 means 20% less. For example, if the written retreat point is 35% HP, a `retreat_hp` of 1.2 makes the bot retreat at 42%. Allowed range: 0.2 to 5.

Dial	What turning it up does
`gem`, `gold`	Camps with more gems / gold look more attractive.
`dist`, `opening_dist`	Walking distance counts more, so near camps win (`opening_` only at game start).
`clear`	Time needed to kill a camp counts more, so quick camps win.
`strength_discount`	Strong (dangerous) camps look less attractive.
`reward`, `shared_small`	Bonus for camps that drop items; how much a small camp shared with allies is worth.
`hysteresis`	Stick with the current target instead of switching back and forth.
`en_detour`, `en_clear`	Allow longer detours and harder camps when clearing "on the way".
`fight_margin`	Need a bigger advantage before starting a fight (more careful).
`retreat_hp`	Retreat earlier (at higher HP).
`aggression`	Chase and attack enemies more.
`recall_gold`	Carry more gold before going home to shop.
`tribute_min`, `tribute_max`	Carry more gems before walking to the altar to deliver them.
`assist_hp`	Need more HP before running to help a teammate.

2. TARGET — 10 weights for "which camp next?"

Every time a champion picks its next camp, it gives each camp a score from the dials above. The TARGET weights then adjust that score with 10 facts about the camp:

gems in it • gold in it • distance • time to clear • time until it respawns • how many others are going for it • damage we would take • how full our bag is • our emblem count • the game round

Each weight is a number from −3 to +3, and 0 means "ignore this fact". A positive weight on *round* makes far camps more attractive late in the game. A negative weight on *damage* avoids camps that hurt. The camp's score is multiplied by $e^{\text{sum of weight} \times \text{fact}}$.

3. FIGHT — 9 weights for "is this fight good?"

Before fighting, the bot estimates its advantage. The FIGHT weights adjust that estimate with: our HP, enemy HP, allies nearby, enemies nearby, whether we are ranged, whether they are ranged, how full our bag is (a full bag means more to lose), and how far we are from our spawn, plus a constant. Range -3 to $+3$, and 0 = off. A negative weight on *bag* makes a champion carrying many gems avoid fights.

4. U — 60 weights for "what do I do right now?"

Every turn, a small decision maker scores 6 possible actions:

keep going • step back • support an ally • fight • walk home • recall (teleport home)

using 10 facts: a constant, our HP, damage per turn from camps, damage per turn from enemies, enemies nearby, allies nearby, an ally in trouble, bag size, distance to spawn, and enemy ranged. Each action has one weight per fact, so $6 \times 10 = 60$ numbers. The highest score wins. All zeros means "do what the hand-written rule says". Range -4 to $+4$.

5. X — 4 extras, and HOT

`tavi_respec` (> 0.5 = Tavi re-buys its stats mid-game). `arbiter_fuel_floor`: the U decision maker only runs when at least this much CPU fuel (25k–60k) is left in the turn, so we never freeze. Two slots are unused. **HOT** (1 number) avoids places we died before; it is fixed at 0 because it made us worse.

Not learned

These are fixed code or fixed per-map plans, and only a human can change them:

- pathfinding, and how attacks and spells are aimed;
- the draft (which champions we pick);
- emblem build order, and the shopping list;
- which map features the bot can "see". If an important fact is missing from the lists above, no amount of training can use it. That is the "model can't express it" case in §6.

See all 102 names and their current defaults: `python harness\params.py --list`.

How training changes the numbers

One round works on one map:

1. Take the current best numbers for that map.
2. Make 8 copies, each nudged by a small random amount. They come in 4 mirrored pairs: one copy goes + and its partner goes −, which makes the comparison fairer.
3. Every copy and the current best play the same opponents (4 mimics + 1 anchor bot) on that map, from both sides. That is about 90 games, run in parallel.
4. If the best copy clearly wins (see below), it becomes the new best for that map.
5. The nudge size adapts: after a success it grows by 15% (explore further), after a failure it shrinks by 15% (fine-tune).

How a copy is judged

Each bot gets a score over its games on that map:

$$\text{score} = \text{win rate} + 0.25 \times \text{gem lead} - 0.01 \times \text{average deaths}$$

Gem lead is (our gems − their gems)/(our gems + their gems), from −1 to +1. So a close loss still scores better than a crushing loss, and dying a lot costs a little.

Example. The current bot wins 6 of 10 games (0.6), has an average gem lead of +0.1, and dies 5 times a game: $0.6 + 0.025 - 0.05 = 0.575$. A copy that wins 7 of 10 with lead +0.2 and 4 deaths scores $0.7 + 0.05 - 0.04 = 0.71$. That is 0.135 better, so it passes the first test.

A copy replaces the current best only if **all four** hold:

(a) its score is at least **0.04** higher on the same games (small differences are luck);
(b) it beats the current best **head-to-head** on that map (at least 50%);
(c) it also scores at least as well against **3 other opponents** that it wasn't trained against (so it didn't just learn one opponent's habits);
(d) it runs out of CPU fuel no more than twice as often (a frozen champion loses games).

Then the new numbers are saved, a full test against every mimic on every map is run, and the result is pushed so all PCs use it. Games are deterministic: the same bots on the same map always play the same game. Any difference between two copies therefore comes from their numbers, not from luck in the game itself.

4 Who we train against

Faithful mimic (`mimic-nyx`)	A copy of a real team. From their recent ladder games we copy their champion picks, emblem builds, first camps on each map, and how often they deliver gems and shop. Their fighting can't be copied (the game data doesn't show it), so a mimic fights like our own bot. Never changed by training, so "our win rate vs Nyx" stays an honest number.
Evolved mimic (`mimx-nyx`)	The same copy, but its numbers are trained to *beat us*. Its picks, builds, first camps and delivery habits stay locked to the real team. If the real teams stop updating their bots, these keep getting harder, so our training doesn't stall. If the real team changes its bot, its evolved mimic starts over.
Anchor bots (`claude-*`, `exploit-*`, `baseline`)	Hand-written bots that attack known weak spots (for example killing our carrier near the altar). They stop old bugs coming back.

We pick opponents that beat us more often than ones we already beat.

5 When a new version goes on the ladder

Only one PC, the **lead**, talks to the platform. If it runs with `--auto-submit`, it uploads the newest trained version when **all** of these are true:

- the trained version is newer than the one on the ladder;
- at least 3 hours have passed since the last upload;
- we lost a ladder game since then, **or** the new version wins at least 5% more against the mimics.

If the new version then does more than 10% worse on the ladder (after 10 ranked games each), the previous version is put back.

The 3 hours count from the last upload's time in `league/ledger.json`, not from when the daemon started. The daemon checks at the start and the end of every round, so after a restart a due upload goes out within seconds. `train status` shows it on the *next upload* line. While the daemon runs, it also stops Windows from going to sleep on its own; you can still put the PC to sleep yourself.

6 What our bot can and can't do

Status on 21 October 2026 (engine 1.0.0). "Tested" means we played old against new on the same games against our copies of the top teams; "no gain" means it didn't win more, so it is switched off.

It can

- **Farm camps.** It picks the next camp by gems, gold, distance and how long the camp takes to kill, with numbers trained for each map. It captures the reward and avoids camps an ally already claimed.
- **Deliver gems.** It walks to an altar once its bag is big enough (bigger when the altar is far, smaller when it is hurt or near the end). Next to an altar it delivers from 5 gems.
- **Shop and recall.** It goes home to buy items when it has gold, and recalls only when nothing can reach it during the recall.
- **Decide fights.** It compares how fast it kills the enemy with how fast the enemy kills it (allies nearby included). It only switches between fighting and fleeing after two turns that agree, so it doesn't flip-flop.
- **Remember fights.** It keeps a short record of recent fights and of each enemy champion type. After three lost exchanges, or a bad record against that type, it stops taking those fights.
- **Finish hurt enemies.** When a teammate reports an enemy champion at half health or less within 4 tiles, it goes to kill it. This is PvP only.
- **Help teammates on its way.** When a teammate calls for help and the fight is at most 4 tiles off its route, it joins.
- **Stop enemy deliveries.** It stuns or attacks an enemy channelling a delivery nearby, which cancels it.
- **Know the current game.** All 13 champions including Heart of Ruin, the Golem camp, and live champion stats read from the engine, so balance patches flow through.
- **Share 8 bytes with its team:** which of the 10 best camps each ally is going for, the last enemy seen (where, how hurt, whether it is delivering), a help flag, and team-wide aggression and bag-size settings.

It can't (yet)

- **Check whether it can survive a camp.** By default it doesn't, so a lone champion can walk into a Golem. *Being tested:* a survival check, and an early-game ramp (easy camps first).
- **See monster damage.** It treats a half-killed camp group as full health. *Next test:* use the live health of the camps it can see.
- **Help with monsters.** It doesn't join an ally fighting a camp next to it. *Next test:* join within 4 tiles.
- **Deliver on the way.** It walks past altars on its route unless it is right next to one. *Next test:* deliver if an altar is at most 3 tiles off the route.
- **Use bag sizes for the new economy.** They were tuned when camps paid 3–5× more gems. *Being tested:* smaller bags.
- **Play skill combos.** Its stun is not timed with its burst. *Tested:* a "stun first, then burst" order; no gain.
- **Dodge or kite.** It only keeps distance when it is ranged and the enemy is melee; it doesn't step out of spells. Melee champions don't dash in (tested; no gain).

- **See far.** It sees only its own vision plus the one shared sighting, and forgets an enemy after 12 rounds.
- **Play Heart of Ruin itself.** It knows how dangerous an enemy Heart of Ruin is at low health, but its draft doesn't pick it and it has no special play for it.

Tried and switched off (measured worse or no better)

Idea	What happened
Full combat state machine	40% fewer deaths, but farmed far less: win rate 0.52 → 0.43.
Help calls with intent (fight / escape / finish / abort)	Win rate fell; removed from the 8 bytes.
Travelling to answer any help call	Abandoned camps; cost more than it saved.
Stepping back instead of walking home	More deaths and fewer gems.
Stun-then-burst combo order	Slightly fewer gems; no gain.
Prism/Vesper "play to the passive" roles	Win rate 0.58 → 0.48.

7 What humans do

Every few hours the lead writes `league/review/<date>.md`. Read it, and write your thoughts in its *Reviewer notes* section. The program keeps that section.

The "gap" column compares our win rate against a team's mimic with our real win rate against that team. A big gap means the mimic is too easy, because the real team does something we didn't copy. Watch their replays.

Losses training couldn't fix (after 6 tries on that map) are listed with a hint:

- *"mimic can't express it"*: we beat their mimic but lost to the real team. Fix the mimic.
- *"model can't express it"*: we keep losing even to the mimic. None of the numbers can fix it, so the bot needs a new feature in the code. Do it on a `feat/` branch, test it, and merge it into `league`. Every PC picks it up on its next round.

8 Trying new ideas: the experiment autopilot

Training tunes numbers; new *behaviour* needs code. Every idea goes into a shared queue, `league/experiments.jso` and the autopilot tests it against the live bot without anyone watching:

1. **Screen:** 7 maps against every mimic, both sides (182 games). The variant and the live bot play the *same* games, and each variant game is compared with the matching live game.
2. **Full check:** every map against every mimic (910 games) plus 70 head-to-head games.
3. **Merge rule** (unchanged): win $\geq$ live, fitness $\geq$ live and head-to-head ≥ 0.45. A human then switches the default on and merges.

Why the screen allows some noise. Any code change, even one that does nothing, uses a little more or less fuel. The bot adjusts its work to the fuel left, so a few decisions flip and the game snowballs differently: a do-nothing change played 2 of 3 maps differently. On 182 games, a strict "at least as good" screen is close to a coin flip, so the screen passes unless the variant is worse by more than one standard error of the paired difference. The full check still decides.

An idea is a set of switches (`knob`/`knob2` values), a git branch with code, or both. Branches are merged onto the live code before testing; one that conflicts is rejected with "needs rebase". A re-run gets a new run number, so its result replaces the old one. Every result, good or bad, stays in the registry: Appendix A is generated from it.

9 The neural net

The bot has a small neural network (109 inputs, 32 hidden, 9 outputs, int8) that can choose the expert (farm, fight, retreat, ...) on hard turns. It is off until a trained one passes the gate.

- **Data:** *data jobs* play logged games of the live bot against the mimics. On 10% of logged turns the bot tries a random sensible expert instead of its rule (switch `EXPLORE`), so the data shows what other choices lead to. Each job stores a small file of (inputs, choice, what happened over the next 40 rounds) on the git branch `ml-data`.
- **Training:** after 6 new data files, a *training job* fits the network by offline reinforcement learning (advantage-weighted regression, `ml/rl/awr.py`): choices that turned out better than expected are copied more strongly. It takes minutes on a CPU, so no GPU is needed.
- **Testing:** each trained network is added to the experiment queue as "switch `ROUTER` on with this network" and goes through the same gate as every other idea.

Why not copy the top teams? We tried. Their full replays record only actions (moves, attacks, spells, purchases); health, gems and gold are not stored, and monsters take turns without an ID. Rebuilding the game state would mean re-writing the game's rules. The top teams stay in training as mimics (Section 4).

10 Two or more PCs

Games are the slow part, and they run on the CPU (a GPU cannot run them). Every extra PC adds games. All coordination goes through git: each PC *claims* an experiment or job (its name and the time, pushed) before starting, so two PCs never do the same one. A claim older than 6 hours is free again. Setup and commands are in the Quick start. `experiments list` and `ml list` show the queues.

11 Maps

The lead follows the platform's current map pool (21 maps in October 2026). New maps are downloaded into `maps/`, and one test game per map records its ID and layout. Maps that leave the pool move to `maps/retired/` and are no longer trained on. Losses on a map we didn't have wait until it arrives.

12 The files that matter

`league/champion.json`	Our best numbers for each map, and the history of improvements. Shared by all PCs.
`bots/main/src/pack/pack.bin`	The same numbers in the format the bot reads. Rebuilt automatically.
`league/mimics/<team>.json`	The evolved mimics' numbers.
`league/ledger.json`	Ladder data: top 10, our games, lost games, uploads. Written by the lead.
`data/mimic/`	The copied teams.
`league/review/`	Reports for humans.
`runs/league/`	This PC's logs and scratch files. Safe to delete.

Never edit the `league/*.json` files or `pack.bin` by hand. If two PCs improve the bot at the same time, the program merges their work. On the same map, a head-to-head game decides which change wins.

13 Commands

Set up a new PC (needs cambc 3.0.1 or newer, the same engine as the ladder):

```
git clone -b league https://github.com/Ahmed-Abdul-Rahim/battlecode-2026a.git
cd battlecode-2026a
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1     (Linux/macOS: bash scripts/
    setup.sh)
.\.venv\Scripts\Activate.ps1
cambc login                       (lead PC only)
```

Run it (Linux/macOS: `scripts/train.sh` with the same words):

```
scripts\train.ps1 start                            normal PC: train and share
scripts\train.ps1 start --lead                     the one lead PC: also reads the ladder
scripts\train.ps1 start --lead --auto-submit       lead that may also upload (only one PC!)
scripts\train.ps1 status                           what is happening
scripts\train.ps1 logs                             live log
scripts\train.ps1 stop
scripts\train.ps1 once --no-push                   one round, no git (for testing)
scripts\train.ps1 sync | maps | bench | review     run one lead step now
scripts\train.ps1 replay [team] [map] [--evolved]   watch the latest model play
scripts\train.ps1 submit                           upload by hand (agree with the team first)

scripts\train.ps1 experiments [list|stop]          the experiment autopilot (Section 7)
scripts\train.ps1 ml [list|stop]                   neural-net data games and training (
    Section 8)
```

Look at games:

```
bctools run main mimic-nyx --maps 00lanes       play our bot vs a mimic
bctools gate main                               quick check vs the baseline bot
cambc view <replay>.repl26a                     watch a replay
cambc matches download <matchId> -o x.repl26a   get a ladder game from the report
```

Rebuild this PDF: `cd docs; latexmk -pdf league.tex; latexmk -c`

14 When something goes wrong

"already running"	Run `scripts\train.ps1 stop`, then start again.
"maps pull failed"	cambc is too old. Run `pip install -U "cambc>=3.0.1"`, then `cambc init --yes .`
"learned.rs and params.py disagree"	Someone changed the list of numbers. Update `harness/params.py` to match.
"build failed"	A broken code change was pulled. Fix it on a branch.
"merge needs a human"	Two people changed the same code. Fix the git conflict by hand.
"no mimics yet"	The lead hasn't synced yet. Training uses the anchor bots meanwhile.
Results differ from the ladder	Check that your cambc is 3.0.1 or newer.
An evolved mimic looks broken	Delete `league/mimics/<team>.json` and push. It starts over.

A Every experiment so far

Generated from `league/experiments.json` by `python harness/experiments.py tex`. Numbers are live→variant on the full pool when it got that far, otherwise on the screen.

tbots/main/src/bc_core/memory.rs', '100644 e7367465603fe08dc0b0385ebeed2 Re-run: the branch conflicted with the live code aft

tbots/main/src/bc_core/memory.rs', '100644 e104209eb453a182007c31e8d53097 Re-run: the branch conflicted with the live code aft

tbots/main/src/bc_core/memory.rs', '100644 033c48b0b4cb7c383c515328e6ee9f Re-run: the branch conflicted with the live code aft

tbots/main/src/bc_core/memory.rs', '100644 033c48b0b4cb7c383c515328e6ee9f Re-run: the branch conflicted with the live code aft

tbots/main/src/bc_core/memory.rs', '100644 033c48b0b4cb7c383c515328e6ee9f Re-run: the branch conflicted with the live code aft

tbots/main/src/bc_core/memory.rs', '100644 033c48b0b4cb7c383c515328e6ee9f Re-run: the branch conflicted with the live code aft

B Technical reference

This appendix is for reviewing the design. Code is quoted from `league` (21 October 2026); file paths are given so you can check the current version.

Determinism. There are no NFAs anywhere: every state machine below is deterministic (a DFA). The engine has no randomness (WASI random returns zeros), so the same two bots on the same map always play the same game. The only randomness in the whole system is the trainer's seeded map pick (§B.6).

B.1 Team channel: the 8 shared bytes (layout v3)

Every champion bot runs separately; the only way to talk is 8 bytes of shared memory the whole team can read and write during the game. Allies act one after another inside a round, so later allies see what earlier ones wrote that round. Codec: `crates/bc-core/src/memory.rs`, `TeamChannel`.

Byte	Bits	Field
0	0–1	version (0 = never written, 3 = this layout)
0	2	parity of the round the leader last wrote
0	3–5	leader slot
0	6	the sighting is an enemy *delivering* (tribute alert)
0	7	help flag ("I'm low and in danger"), cleared each round by the leader
1	0–7	team aggression (0–255), or a camp respawn broadcast (TEAM_RESPAWN)
2	0–7	team bag threshold (gems), or the respawn round (TEAM_RESPAWN)
3–7	0–20	camp claims: 6 slots, packed base 11
3–7	21–34	last enemy sighting: x (7 bits), y (7 bits)
3–7	35–36	sighting age: round mod 4
3–7	37–38	sighted enemy health in quarters (<25%, <50%, <75%, more or unknown)
3–7	39	spare

Camp claims are 6 numbers 0–10 (0 = none, k = priority index $k-1$, so only the 10 best camps of the draft's priority list can be claimed). $11^6 = 1{,}771{,}561 < 2^{21}$, so they fit in 21 bits as one base-11 number:

```
let mut packed = 0u64;
for c in self.claims.iter().rev() {              // slot 5 first, slot 0 last
    let x = c.map_or(0, |i| u64::from(i.min(MAX_CLAIM_INDEX)) + 1);
    packed = packed * CLAIM_BASE + x;            // CLAIM_BASE = 11
}
let v = packed | s << CLAIM_BITS;                // s = sighting x | y<<7 | r<<14 | hp_q<<16
b[3..8].copy_from_slice(&v.to_le_bytes()[..5]);
```

Leader protocol. The first ally to act in a round sees a stale parity bit and becomes the leader: it clears the help flag and drops a sighting whose age reached 3 (age 3 can't be told from −1 with 2 bits). **Who writes what:** every champion posts the nearest enemy it sees, with its health quarter (`post_sighting_hp`); a fresh tribute alert is not overwritten for 2 rounds; claims are rewritten every turn by each slot.

History of the layout. v1: 4-bit claims (15 camps), no health. v2: 3-bit claims (7 camps) plus help calls with intent (fight/escape/finish/abort) and time left; measured worse, removed. v3: the table above.

B.2 Draft memory: the per-match parameters (1,024 bytes)

The draft bot sees the whole map with 10× fuel and writes `ThetaMatch` once; every champion reads it on its first turn (`memory.rs`, `docs/memory-layout.md`). Main parts: team size, role per pick, the camp priority list (up to 15 groups with scores; the claims above index into it), FiLM modulation for the learned router, **32 knob bytes**, champion vectors and kinds for both teams, a map-descriptor cache and a CRC. The knobs used by this document:

Knob (index)	Value	Meaning
TRIBUTE_MIN_GEMS (2)	20	smallest bag worth a trip
TRIBUTE_MAX_GEMS (3)	120	always deliver at this bag
CAMP_FEASIBLE (23)	0	bits: 1 survival check, 2 e 16 altar on way
COMBAT_FSM (31)	195	bits: 64 hysteresis, 128 he off)

B.3 The trained vector θ and `pack.bin`

θ has 102 floats per map (`harness/params.py`, `bots/main/src/learned.rs`):

Block	Index	Search space	Bounds
W multipliers	0–17	log (bot uses e^x)	$[\log 0.2, \log 5]$
TARGET weights	18–27	linear	$[-3, 3]$
FIGHT weights	28–36	linear	$[-3, 3]$
HOT	37	fixed	0
U (6 actions × 10 facts)	38–97	linear	$[-4, 4]$
X: Tavi re-spec, fuel floor, FSM alert, FSM assist	98–101	linear	$[0, 1]$, $[25, 60]$, $[0, 12]$, $[0, 2]$

Training perturbs the search-space vector; `params.to_bot` clips to the bounds and applies e^x to W. `league/champion.json` stores the search-space vector per map; `pack.bin` stores bot-space floats:

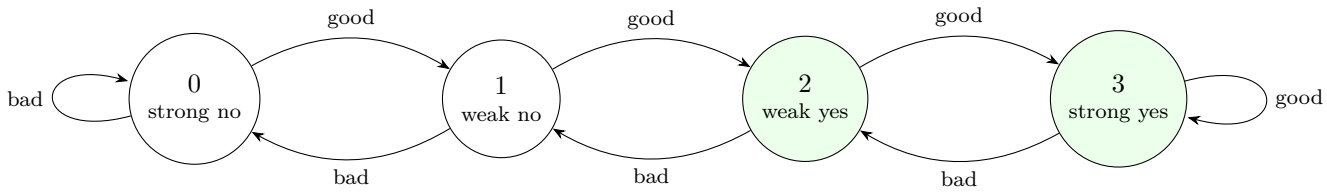
```
"BCPK" u16 version=1 u16 n_entries
per entry:   u32 map_fingerprint  u16 n_sections
per section: u8 tag  u32 length  bytes
  tag 1  theta     f32[] (may be shorter than 102: the rest = learned.rs defaults)
  tag 2  hotspots  (x u8, y u8, weight u8)[]
  tag 3  opening   n_sides u8; per side: spawn x,y u8, then 13 kinds x 4 group indices (255 = end)
  tag 4  draft     3 team sizes x 5 picks (255 = end)
  tag 5  build     13 kinds x 12 emblems as family*3+stat (255 = end)
```

The map fingerprint is an FNV-1a hash of width, height, walls, holes and altars (`world::fingerprint`); the lead registers each new map's fingerprint by playing one debug game (`register_map`). Our champion pack only has tag 1; mimic packs also carry tags 3–5 copied from the real team.

B.4 State machines

2-bit saturating counter (`bc_core::combat::Ctr2`)

Four states, like a branch predictor. "Yes" = states 2 and 3, so one contrary turn never flips a strong answer:



Start state: 1. A turn is *good* when `advantage` $\geq 1 +$ margin **and** the history allows it (next part).

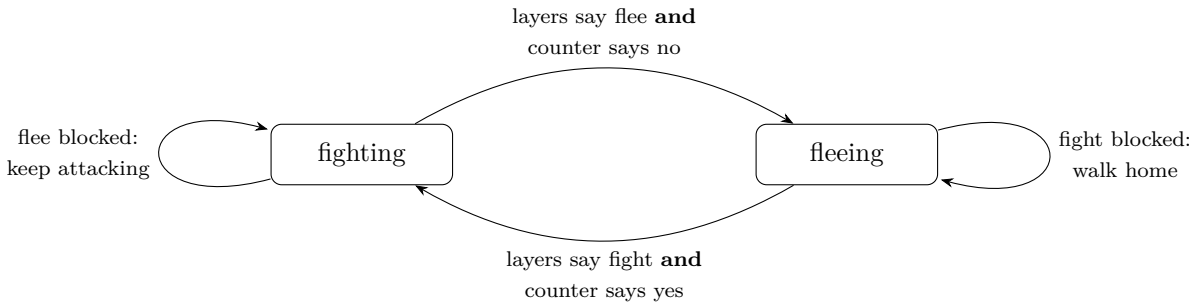
Fight history

```
pub struct History {
    pub results: u8,            // last 8 exchanges, newest in bit 0: 1 = we dealt >= we took
    pub len: u8,                // how many are recorded (max 8)
    pub vs_kind: [Ctr2; 13],    // per enemy champion type: fights go well / badly
}
// history allows a fight unless: vs_kind[enemy] is "strong no", or the last 3 exchanges were all
    lost
let history_ok = !hist.bad_against(e.kind) && hist.recent_losses(3) < 3;
```

An *exchange* is one turn near an enemy champion (within its range + 2): damage we dealt is the drop in the watched enemy's health since our last turn; damage we took counts only when an enemy champion could hit us. Turns with neither are not recorded.

Hysteresis gate (live: COMBAT_FSM bit 64)

The other layers still choose this turn's action; the gate only blocks a switch between fighting and fleeing that the counter doesn't support. The memory is one value, `fsm_fighting` $\in$ {none, fighting, fleeing}, reset when no enemy is in view:

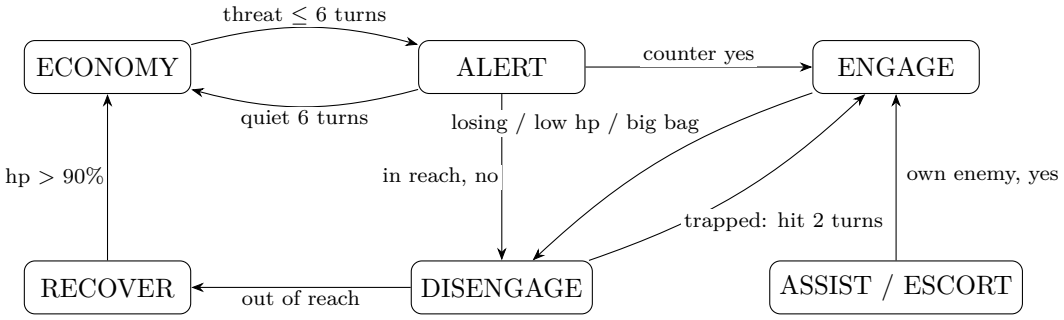



```
let kept = match self.fsm_fighting {
    // was fighting, told to flee: only if the counter agrees (safety overrides always win)
    Some(true) if flees && self.fsm.fight.yes() && !matches!(mode, Mode::Override(_)) =>
        if ranged_vs_melee { Intent::Kite(e.pos) } else { Intent::Engage(e.pos) },
    // was fleeing, told to fight: only if the counter agrees
    Some(false) if fights && !self.fsm.fight.yes() => Intent::WalkHome,
    _ => intent,
};
```

Points to criticise: the counter is updated from the same advantage estimate the layers use, so it filters noise but can't correct a wrong estimate; the gate never overrides the low-health safety rule.

Full combat state machine (COMBAT_FSM bit 1 alone: off)

Kept in `crates/bc-core/src/combat.rs` for future work; measured worse (§6).



States have minimum dwell times (ENGAGE 2 turns); ALERT blocks captures and deliveries when a threat can arrive before the channel ends. Its cost was economic: champions spent too many turns not farming.

B.5 Decision formulas

Fight advantage (`crates/bc-brain/src/router.rs`):

```
let our_dps   = (ctx.my_dps + ctx.ally_dps_near).max(1.0);          // live stats, items included
let their_dps = (e.dps * f32::from(ctx.enemies_in_range.max(1))).max(1.0);
let advantage = (ctx.hp / their_dps) / (e.hp / our_dps).max(0.1)  // time to kill us / time to kill
     them
            * ctx.fight_bias.exp();                                  // learned FIGHT weights
// fight if advantage >= 1 + margin - bounty + risk
//   margin = fight_margin% x (255 - aggression) / 127;  bounty 0.2 if the enemy carries >= 20 gems
//   risk 1.0 if we carry a big bag;  enemy dps x (1 + 0.5 x missing hp) for Heart of Ruin
```

Camp choice (`bots/main/src/bot.rs`, `choose_target`), per camp group:

$$\text{score} = \frac{\text{value}}{\text{dist}\cdot w_{\text{dist}} + \text{clear}\cdot w_{\text{clear}} + \max(0,\text{eta}-\text{dist}) + 5} \times e^{\,\text{TARGET}\cdot\phi} \times (1.3\, w_{\text{hyst}} \text{ if current target})$$

where

- value $= (\text{gems}\cdot w_{\text{gem}} + 0.6\,\text{gold}\cdot w_{\text{gold}}) \,/\, (1 + \text{max strength}/100 \cdot w_{\text{strength}})$;
- clear = group HP/our live damage (turns to kill it alone);
- eta = rounds until it respawns.

The w are the trained W dials. *Missing today (being tested):* survival (kill time $> 3\times$survival time $\Rightarrow \times 0.1$), the early ramp ($\div\, 1 + \max(0, \text{clear} - 20)/20$) and live camp HP.

Delivering gems (`policy::should_tribute`): deliver when the bag reaches TRIBUTE_MAX, or bag $\geq$ TRIBUTE_MIN + dist to altar $\times$ scale/10, halved when hurt, threatened or after round 1800; within 2 tiles of an altar the requirement is at most 5 gems.

Kill assist (bit 2): if no enemy champion is within 4 tiles, we carry less than a big bag, our health is over 40%, and the shared sighting is at most 1 round old, within 4 tiles and at $\leq 50\%$ health (quarters 0–1) $\Rightarrow$ engage it.

Help on my way (bit 128): the goal is where the current intent leads (camp centre, nearest altar or home); we answer the 1-bit help flag only if $\text{dist}(me, p) \leq 10$ and $\text{dist}(me, p) + \text{dist}(p, goal) - \text{dist}(me, goal) \leq 4$ (Chebyshev distances).

B.6 League daemon logic

```
def cycle():                          # harness/league.py, one round
    integrate()                       # git fetch + rebase; champion.json is merged per map
    seed_maps(); apply_champion()     # every pool map has a theta; pack.bin follows champion.json
    maybe_release()                   # lead + --auto-submit: upload if due (also at the end)
    if lead: watch(); refresh_maps(); sync()   # ladder games, map pool, top-10 mimics
    if every 3rd round and no map still needs bootstrap: evolve_mimic()
    else: train()                     # 8 antithetic ES copies of one map's theta vs 5 opponents
    publish(champion.json, pack.bin)  # on a promotion or every 30 min (try counts)
    maybe_release()

def release_decision(ch, led, now):   # upload v if all hold
    newer = ch["version"] > live_version
    gap   = now - last_upload_time >= 3 * 3600
    why   = any loss since the last upload  or  bench(v) >= bench(live) + 0.05

def map_pressure(led, now):           # used when no open loss of the live version is queued
    for loss in led["losses"] (except needs-human):
        w = exp(-(now - loss.time) / 24h) * (2 if the live version lost it else 1)
        pressure[loss.map] += w
# pick map ~ (pressure[m] + 0.1 * max pressure + 0.05) * (1.1 - local win rate on m)
```

Promotion gate: a copy replaces the champion's θ on that map only if its fitness (win $+0.25$ gem lead -0.01 deaths) beats the champion by 0.04 on the same games, it wins the head-to-head (≥ 0.5), it is at least as good on 3 fresh opponents, and it runs out of fuel at most twice as often.

Merging two PCs: `merge_champion` is a 3-way merge per map; if both sides changed the same map, a head-to-head on that map decides; promotion history is unioned and renumbered.